```
data['amount_normalized'] = (data['amount'] - data['amount'].mean()) / data['amount'].std() data = pd.get_dummies(data, columns=['transaction_type'], drop_first=True) # Step 3: Train-Test Split
X = data.drop(['is_fraud'], axis=1)
y = data['is_fraud']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42) # Step 4: Train Isolation Forest
model = IsolationForest(contamination=0.01, random_state=42) model.fit(X_train)

# Step 5: Predict and Evaluate
predictions = model.predict(X_test)
predictions = [1 if p == -1 else 0 for p in predictions]
print(classification_report(y_test, predictions)) auc = roc_auc_score(y_test, predictions)
print(f"AUC-ROC Score: {auc}")

# Step 6: Real-Time Prediction Example def predict_transaction(transaction):
    transaction_df = pd.DataFrame([transaction]) transaction_df['amount_normalized'] = (transaction_df['amount'] - data['amount'].mean()) / data['amount'].std() transaction_df = pd.get_dummies(transaction_df, columns=['transaction_type'], drop_first=True) prediction = model.predict(transaction_df) return "Fraud" if prediction[0] == -1 else "Legitimate"

# Example Usage
new_transaction = {
    'amount': 1200,
    'transaction_type': 'online',
    'user_age': 35,
    'device': 'mobile'

}

print(predict_transaction(new_transaction))
```

Project 10: Chatbot for Customer Support

Objective:

To create an intelligent chatbot capable of handling customer queries efficiently, reducing response times and streamlining customer support operations. The chatbot will leverage advanced Natural Language Processing (NLP) techniques to understand and respond to user intents effectively.

Detailed Plan:

1. Dataset Selection and Preparation:

- Use publicly available datasets like: